OPSL

Offline PM Scripting Language

Language Design & Implementation Document · v0.7

`Runtime: Node.js · Extension: .opsl · CLI: opsl run script.opsl`

1. Philosophy & Core Idea

OPSL is **not a traditional programming language.** No compiler, no type system, no standard library. Its sole purpose: call functions from any real language in a clean, flat, readable style — and compose them with pure OPSL logic.

Key Principles:

- Every operation is a call to a library function — language library or OPSL library
- Code is always linear — top to bottom, no nesting, no branches in the script itself
- The dot (.) is reserved entirely for libraryName.functionName — always, everywhere
- [lang] tags appear ONLY on IMPORT lines — never at any call site
- The executor knows which runner to use because the library was already imported with a tag
- No compilation — the runtime reads and executes .opsl files line by line
- Loops, conditionals, and control flow are OPSL library functions — no built-in keywords
- Comments use // — C-style, full line or inline

2. Complete Syntax Reference

2.1 Comments

Comments start with // and run to end of line. Stripped before any parsing.

```
// Full-line comment
result = csvtools.read(path)  // inline comment — also valid
```

2.2 IMPORT — The Only Place a [lang] Tag Appears

All libraries are loaded with `IMPORT`. The `[lang]` tag appears **only here** — it tells OPSL which language runner to use. After import, the library is called by name alone — no tag ever again.

```
IMPORT [py] csvtools  // Python library — called as csvtools.fn()
IMPORT [js] formatter  // JavaScript library — called as formatter.fn()
IMPORT [sh] gittools  // Bash library — called as gittools.fn()
IMPORT [java] datatools  // Java library — called as datatools.fn()
IMPORT [c] mathlib  // C library (pre-compiled) — called as mathlib.fn()
IMPORT [opsl] loop  // OPSL library — called as loop.fn()
```

- The [lang] tag is recorded by the executor at import time — library name is mapped to its runner
- Every call site uses only the library name — the executor looks up which runner to use
- The folder name is the library name — it cannot be changed or aliased

- Resolution: .local-offlinepm/ first, then ~/.opsl/libs/
- All imports resolved before execution begins

2.3 Variables

Plain name = value. No type keywords. Variables live in OPSL scope — subject to the closure rules in §4.

```
filePath = "./data/sales.csv"
label = "Q1 Report"
threshold = 42
records = csvtools.read(filePath)  // no [py] here — executor knows csvtools is Python
```

2.4 The Dot Operator — libraryName.functionName

The dot is **reserved entirely** for libraryName.functionName. This applies equally to language libraries and OPSL libraries. The language is never repeated at the call site — the import already declared it.

```
IMPORT [py] csvtools
IMPORT [js] formatter
IMPORT [opsl] mathutils

records = csvtools.read(filePath)  // Python — no [py] tag here
html = formatter.toHTML(records)  // JavaScript — no [js] tag here
sq = mathutils.square(threshold)  // OPSL library — same syntax
```

2.5 All Call Forms

Form	Syntax	When to use
Single assign	`x = library.fn(a, b)`	One return value
Multi-assign	`x, y = library.fn(a)`	Multiple return values
Bare call	`library.fn(a, b)`	Side effects, no return needed
Local DEFINE call	`x = myFunc(a, b)`	Call a locally-defined function (no dot — no library)

```
IMPORT [py] csvtools
IMPORT [py] stats
IMPORT [js] mailer

data = csvtools.read(filePath)  // Python runner used
minimum, maximum = stats.bounds(data)  // Python runner used
mailer.send("team@co.com", "Report", data)  // JavaScript runner used
report = buildReport(data, label)  // local DEFINE — no dot
```

2.6 DEFINE — Inline Functions

DEFINE creates a function local to the current .opsl file. Functions have full closure access to global scope. See §4 for the complete scoping rules.

```
IMPORT [py] csvtools

DEFINE processFile(path) {
  raw = csvtools.read(path)  // no [py] — executor knows csvtools is Python
  clean = csvtools.clean(raw)
```

```
  counter = counter + 1              // writes global directly
  return clean
}
```

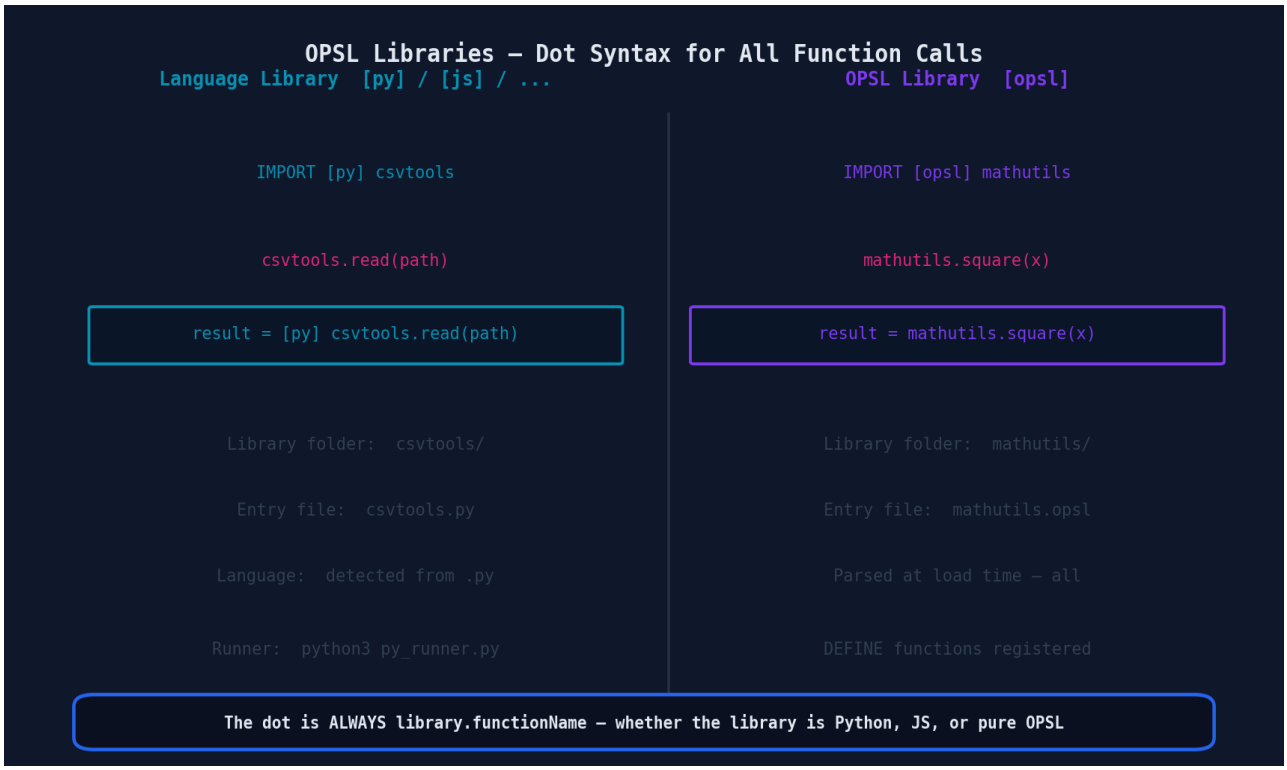
```
IMPORT [py] stats

DEFINE analyseData(data) {
  mean, stddev = stats.distribution(data)
  return mean, stddev
}
```

2.7 OPSL Libraries

Any .opsl file can be a library. Its functions are called with the same dot syntax as every other library. `IMPORT [opsl]` tells the runtime to parse the .opsl file and register its DEFINE functions — no runner involved.

```
// mathutils/mathutils.opsl  — OPSL library file
IMPORT [py] math

DEFINE square(x) {
  result = math.multiply(x, x)
  return result
}

DEFINE cube(x) {
  sq = mathutils.square(x)  // OPSL lib calling itself
  result = math.multiply(sq, x)
  return result
}
```

```
// Using the OPSL library — identical syntax to any other library
IMPORT [opsl] mathutils

sq = mathutils.square(5)
cu = mathutils.cube(3)
```



2.8 Loops and Control Flow via OPSL Library Functions

OPSL has **no built-in loop or conditional keywords.** Loops, maps, filters, and conditionals are OPSL library functions — written once in a real language, wrapped as an OPSL library, and used like any other library. Anyone can define their own control flow. There is no special syntax for this — it all falls out of the existing language.

To pass a function as an argument, use the same **libraryName.functionName** dot syntax used everywhere else. For locally-defined DEFINE functions, use the bare function name (no dot — no library). The executor detects that the argument is a function reference and wraps it via the `OPSL_FNREF:` protocol so the language runner can call back into OPSL per-iteration.

loop/loop.opsl — example control-flow OPSL library

```
// loop/loop.opsl — a user-defined OPSL control-flow library
// Its DEFINE functions wrap Python iteration internally
IMPORT [py] _loopengine

DEFINE forEach(fn, list) {
  _loopengine.runForEach(fn, list)  // Python drives iteration, calls back per item
}

DEFINE times(fn, n) {
  _loopengine.runTimes(fn, n)
}

DEFINE ifThen(condition, trueFn, falseFn) {
  _loopengine.runIf(condition, trueFn, falseFn)
}
```

Using the loop library in a script

```
IMPORT [py] csvtools
```

```
IMPORT [opsl] loop
IMPORT [opsl] transforms

records = csvtools.read("./data/raw.csv")

DEFINE processItem(item) {
  clean = transforms.cleanRow(item)  // OPSL lib fn — dot syntax
  valid = transforms.isValid(clean)
  loop.ifThen(valid, transforms.addToOutput, transforms.logSkipped)
}

// Pass local DEFINE fn by bare name, library fns by dot syntax
loop.forEach(processItem, records)  // bare name — local DEFINE
loop.forEach(transforms.cleanRow, records)  // dot syntax — OPSL lib fn
loop.times(csvtools.flush, 3)  // dot syntax — language lib fn
```

Under the hood: when the Python runner inside `_loopengine` needs to invoke `processItem` per iteration, it writes `OPSL_CALLBACK:processItem|item` to stdout. worker-base.js intercepts it, the executor runs `processItem(item)`, and sends the result back as `OPSL_RETURN:result`. The runner receives it and continues the loop. The OPSL script sees none of this — it just calls `loop.forEach` and gets a result.

2.9 Complete Syntax Grammar

Construct	Syntax
Import — only place [lang] appears	`IMPORT [ext] libraryName`
Import OPSL library	`IMPORT [opsl] libraryName`
Variable assignment	`varName = value`
Library call — single return	`var = library.function(args)`
Library call — multiple return	`v1, v2 = library.function(args)`
Library call — bare (no return)	`library.function(args)`
Local DEFINE call	`var = functionName(args)  — no dot, no tag`
DEFINE function	`DEFINE name(params) { ... }`
Return single value	`return varName`
Return multiple values	`return val1, val2, val3`
Pass library fn as arg	`lib.fn(otherLib.otherFn, data)  — dot syntax`
Pass local DEFINE as arg	`lib.fn(myLocalFunc, data)  — bare name only`
Comment	`// text — full line or inline`
String literals	`"double"  or  'single'  — both valid`
Inline number literals	`library.fn(5, 10.5)  — valid anywhere`

3. Example Programs

3.1 Hello World

```
IMPORT [py] console

console.print("Hello, World!")
```

3.2 Multi-Return

```
IMPORT [py] mathtools

minimum, maximum, mean, stddev = mathtools.describe(42, 7, 99, 3, 55)
mathtools.printSummary(minimum, maximum, mean, stddev)
```

3.3 Mixed Python + JavaScript

```
IMPORT [py] csvtools
IMPORT [py] stats
IMPORT [js] renderer

records = csvtools.read("sales.csv")
summary = stats.summarise(records)
html = renderer.toHTML(summary)
renderer.saveFile(html, "report.html")
```

3.4 Loops and Transforms via OPSL Libraries

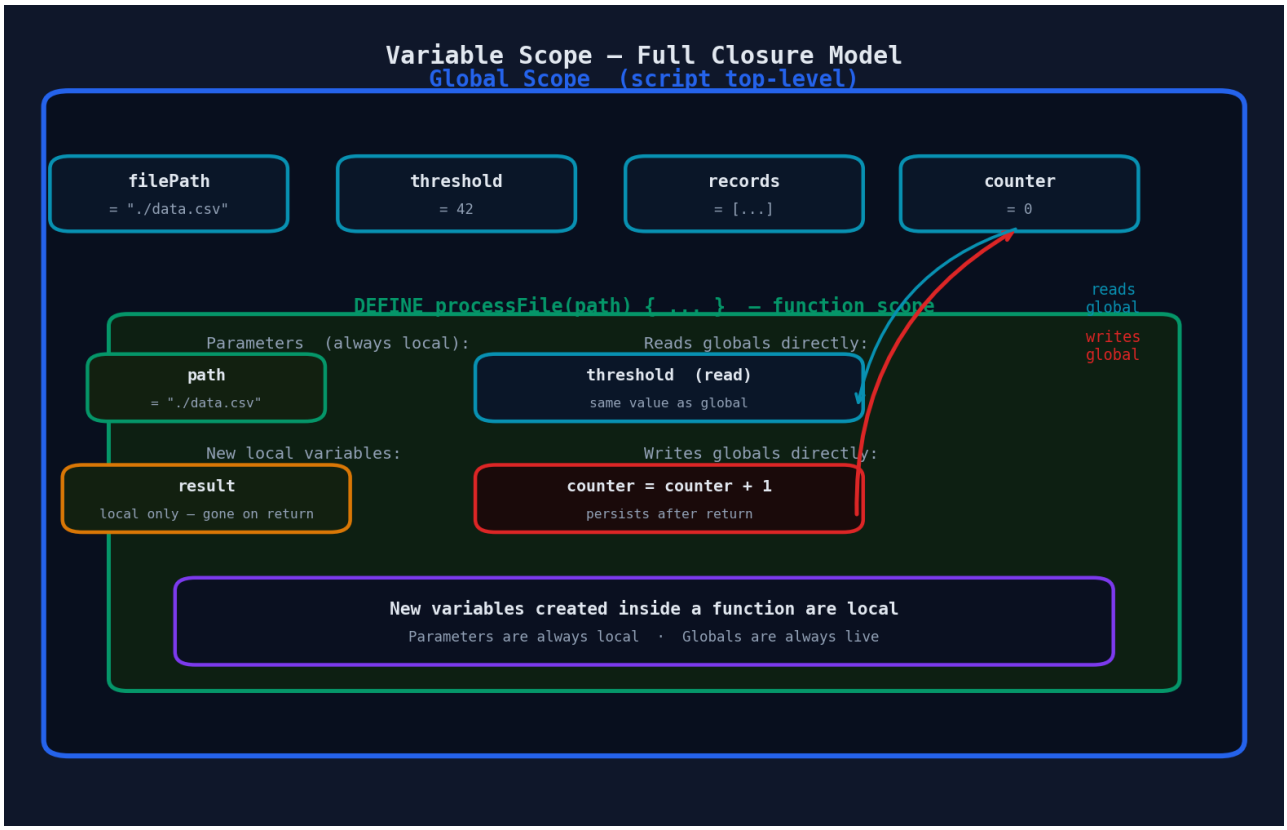
```
IMPORT [py] csvtools
IMPORT [opsl] loop
IMPORT [opsl] transforms

records = csvtools.read("./data/raw.csv")

// Pass OPSL library functions by dot syntax
cleaned = loop.map(transforms.cleanRow, records)
filtered = loop.filter(transforms.isValid, cleaned)

csvtools.write(filtered, "./data/clean.csv")
```

3.5 Full Pipeline with Globals, Closures, and Loops

```
IMPORT [py] csvtools
IMPORT [py] stats
IMPORT [js] mailer
IMPORT [opsl] loop

processed = 0
errors = 0

DEFINE processFile(path) {
  raw = csvtools.read(path)
  clean = csvtools.clean(raw)
  processed = processed + 1      // writes global directly
  return clean
}

DEFINE buildReport(data) {
  mean, stddev = stats.distribution(data)
  report = stats.formatReport(mean, stddev, processed, errors)
  return report
}

files = csvtools.listFiles("./data/")
loop.forEach(processFile, files)  // local DEFINE — bare name

allData = csvtools.mergeAll()
report = buildReport(allData)
mailer.send("team@company.com", "Weekly Report", report)
```

4. Variable Scope — Closure Model

OPSL uses a **full closure model.** Every DEFINE function can read and write the global script scope directly.



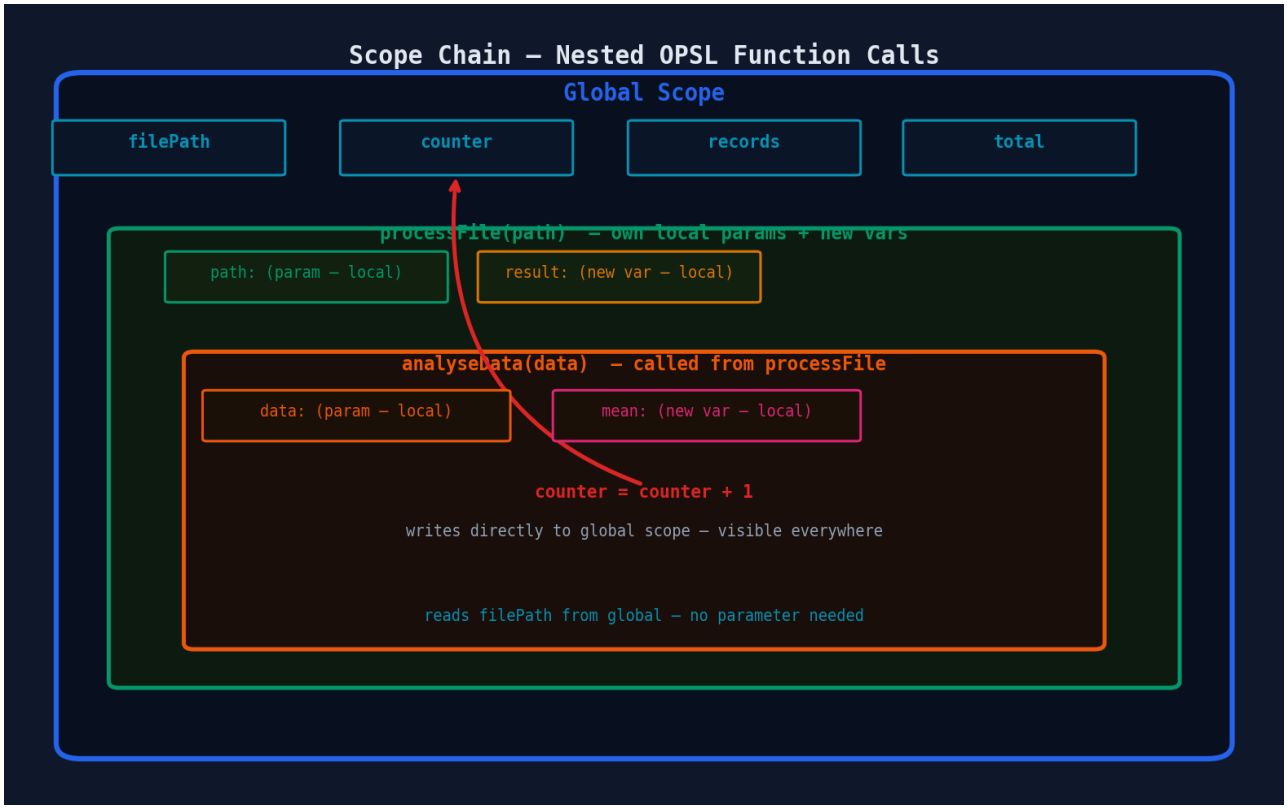
4.1 The Rules

- Parameters are always local. Modifying a parameter never affects anything outside.
- New variables created inside a function are local. They disappear when the function returns.
- Global variables are always visible and writable. A function reads or writes any global directly. Changes persist after the function returns.
- No shadowing — **assigning to a name that exists in global scope overwrites it.** Use a different name for a local copy.
- OPSL library functions (from .opsl files) run in fully isolated scope — they cannot access the calling script's globals.

4.2 Scope in Practice

```
IMPORT [py] csvtools
IMPORT [py] console

counter = 0
filePath = "./data.csv"

DEFINE incrementAndProcess(path) {
  counter = counter + 1            // reads AND writes global counter
  raw = csvtools.read(path)  // raw is local — brand new name
  return raw
}

DEFINE reportStatus() {
  console.print(counter)  // reads global — no parameter needed
  console.print(filePath)  // reads global — no parameter needed
}
```

```
result = incrementAndProcess("./data.csv")
reportStatus()  // counter is now 1 in global scope
```



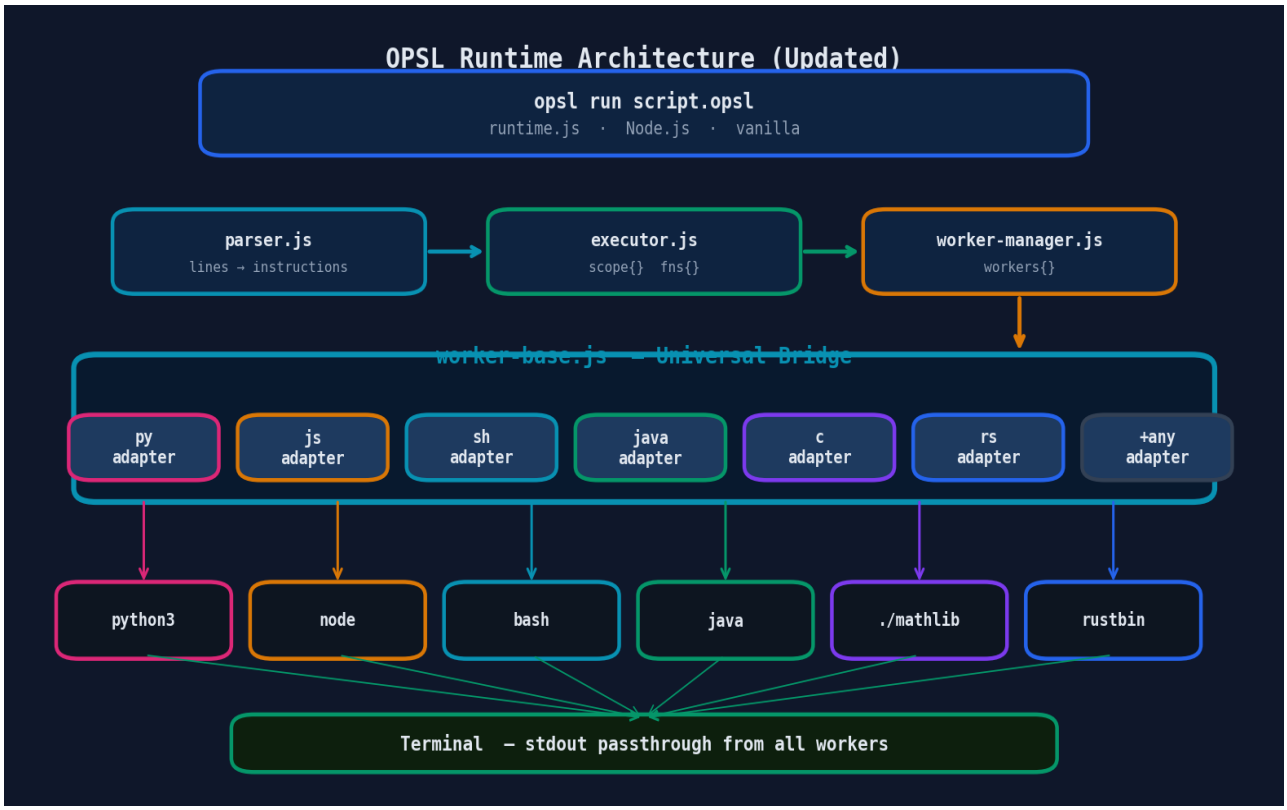
4.3 Scope in the Executor — Backend Detail

The executor maintains `scope = {}` — a single flat object. When a DEFINE function is called:

- `params` go into a function-local object — always local, never touch global scope
- Writes inside the function: if the name exists in global scope → overwrite global; if brand new → create local
- When the function returns, the local object is discarded. Changes to global names persist in `scope`
- OPSL library functions get a completely fresh empty scope — fully isolated from the calling script

5. Runtime Architecture

OPSL runs as a single Node.js process coordinating one persistent language runner per language tag. The executor maintains an `importRegistry` that maps every library name to its language — so call sites never need a tag.



5.1 File Structure

Project layout

```
opsl/
  runtime.js            ← entry: reads file, runs parser then executor
  parser.js             ← converts .opsl lines into typed instruction objects
  executor.js           ← scope, dispatch, import registry, closure model
  worker-manager.js     ← spawns runners using adapters, routes calls
  worker-base.js        ← universal bridge: handles ALL protocol logic
  adapters/
    py-adapter.js       ← ~5 lines: start command for python runner
    js-adapter.js       ← ~5 lines: start command for node runner
    c-adapter.js        ← ~5 lines: start command for compiled binary
  runners/
    py_runner.py        ← reads OPSL_CALL:, writes OPSL_RESULT:
    js_runner.js
    sh_runner.sh
  languages/
    py.json  js.json  sh.json  java.json  c.json
```

5.2 parser.js — No [lang] on Call Lines

Pure function. Lines in, instruction objects out. The parser **never sees a [lang] tag on a call line** — it only parses libraryName.function(args) and records the library name. Which runner to use is resolved later by the executor via its import registry.

- 1. Blank / comment → skip
- 2. DEFINE start → collect body until }
- 3. IMPORT → extract [lang] tag + library name (only line type with a [lang] tag)
- 4. return → extract return value tokens
- 5. Assign with dot call → targets, libraryName, fn, args

- 6. Bare dot call → libraryName, fn, args
- 7. Assign from local DEFINE → targets, fn name, args (no dot, no library)
- 8. Bare local DEFINE call → fn name, args

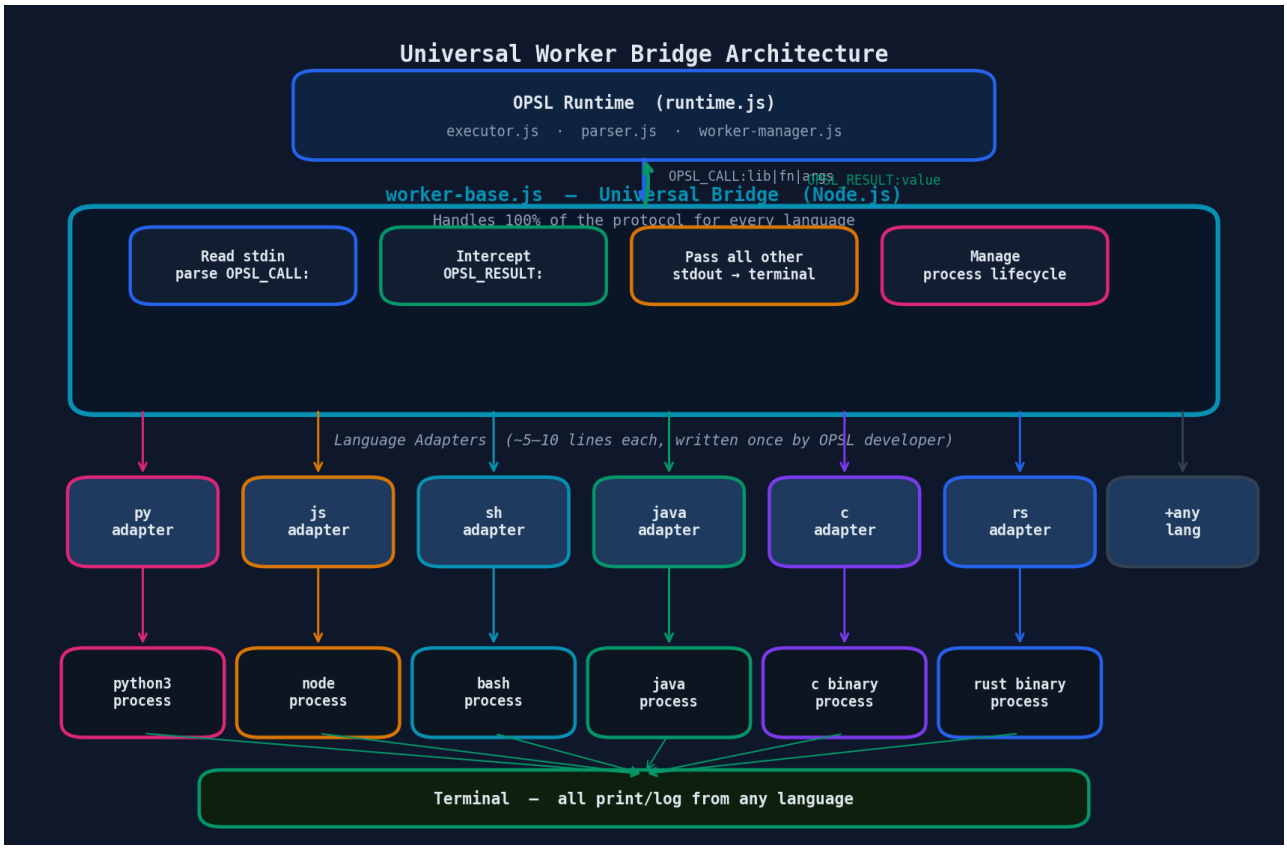
5.3 executor.js — Import Registry

At IMPORT time the executor records: `importRegistry["csvtools"] = "py"`. When a call to `csvtools.read()` is dispatched, the executor looks up `"py"` and routes to the Python runner. The parser never needs language information on calls.

- `importRegistry` — libraryName → lang tag, populated at IMPORT time
- `scope` — flat global variable map
- `functions` — DEFINE registry (local to this script)
- `libraries` — OPSL library function registries (from [opsl] imports)
- Arg resolution: if arg matches a known function → wrap as `OPSL_FNREF:library.fn` (dot syntax) or `OPSL_FNREF:localFn` (bare name)

6. Universal Worker Bridge

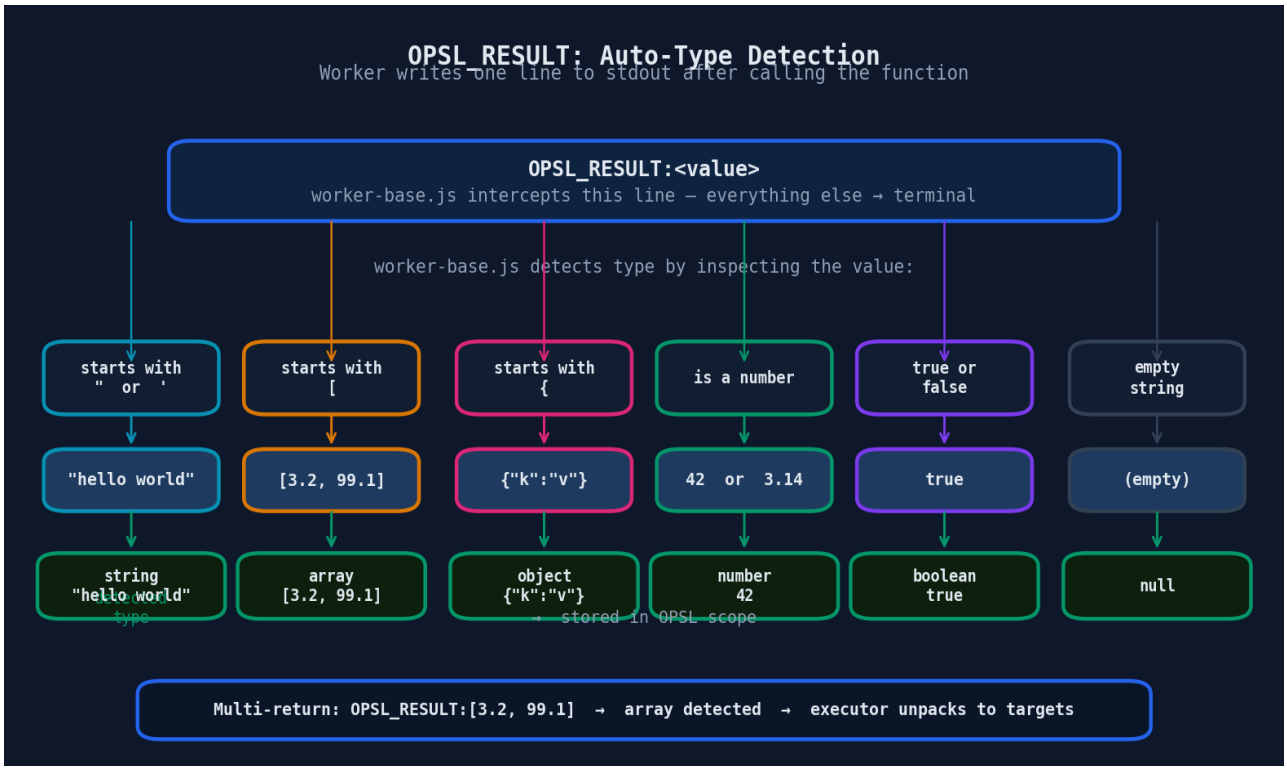
worker-base.js is written once and never changes when adding languages. Each language needs only an adapter (~5 lines) and a runner (~10 lines).



6.1 Protocols

Complete protocol

```
// OPSL → Runner stdin:
OPSL_LOAD:csvtools|/path/to/csvtools/csvtools.py
OPSL_CALL:csvtools|read|./data.csv
OPSL_CALL:stats|bounds|1|2|3|4|5

// Runner → OPSL stdout (return values):
OPSL_RESULT:42
OPSL_RESULT:"hello"
OPSL_RESULT:[3.2, 99.1]
OPSL_RESULT:

// Runner → OPSL stdout (callback for loops/control flow):
OPSL_CALLBACK:processItem|item_value

// OPSL → Runner stdin (callback result):
OPSL_RETURN:callback_result

// Runner → OPSL stdout (error):
OPSL_ERROR:FileNotFoundError: ./data.csv not found
```



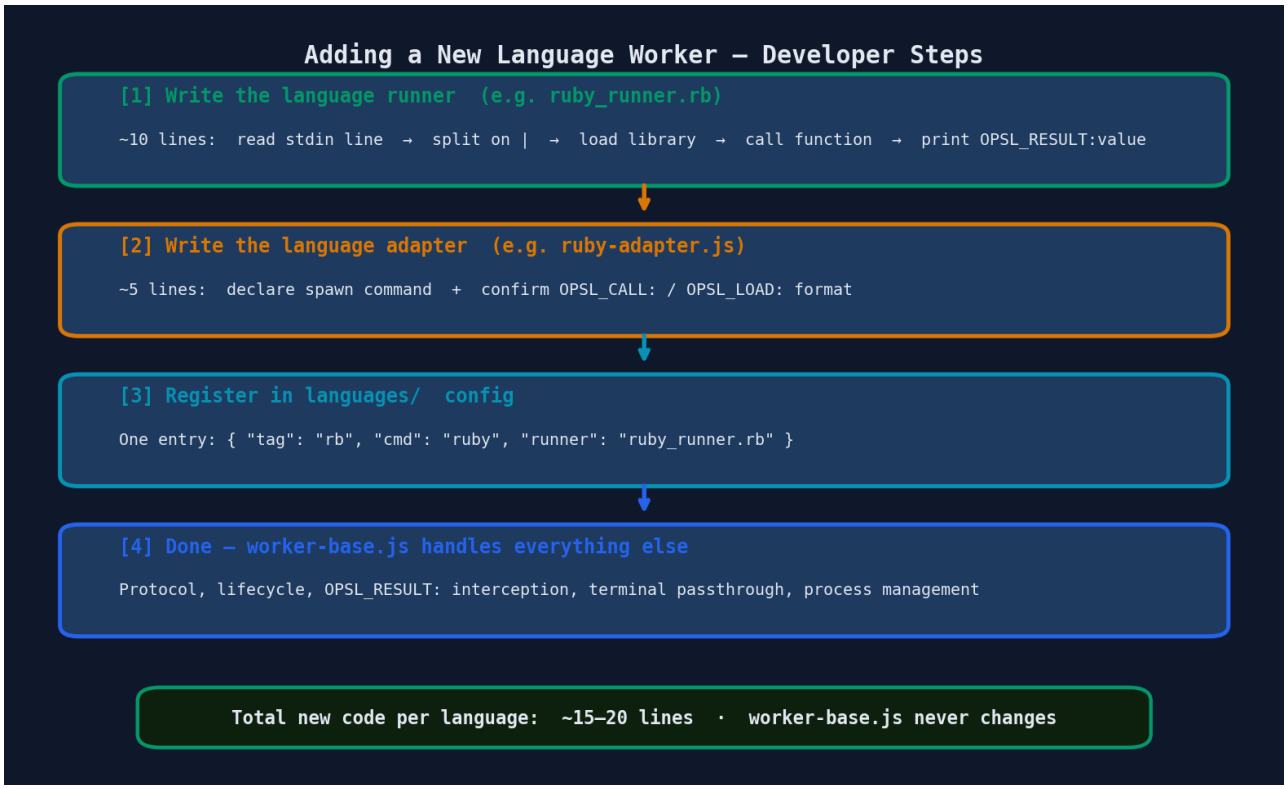
6.2 Adapter

```
// adapters/py-adapter.js  — complete, ~5 lines
module.exports = {
  cmd:    "python3",
  runner: "runners/py_runner.py",
};
```

6.3 Runner Contract

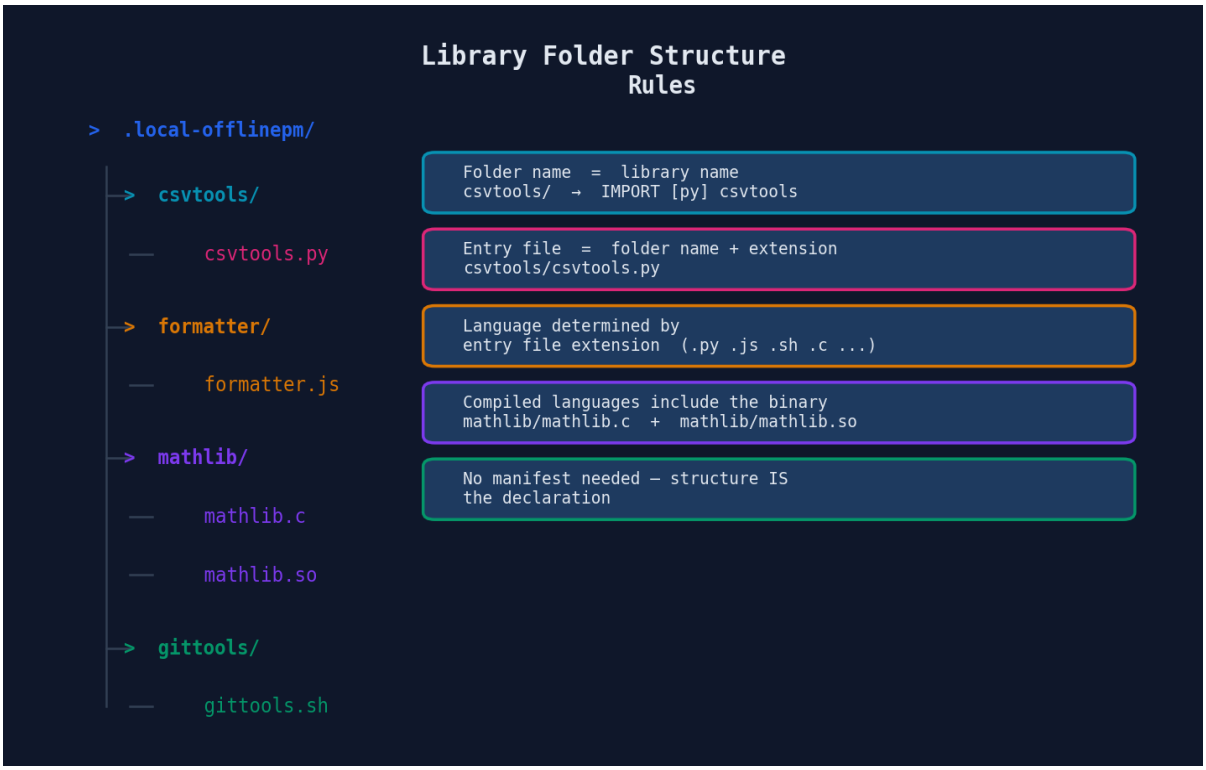
```
// Every runner must implement:
```

```
// 1. Loop: read one line from stdin
// 2. OPSL_LOAD:lib|path  →  load the library file
// 3. OPSL_CALL:lib|fn|arg1|arg2  →  call fn, print OPSL_RESULT:value
// 4. If any arg is OPSL_FNREF:name  →  that arg is a function reference
//    Per iteration: print OPSL_CALLBACK:fnRef|item
//    Read next stdin line as OPSL_RETURN:value  →  use as result
// 5. On exception: print OPSL_ERROR:message
// 6. Repeat from step 1 forever
```



7. Library Structure & Resolution

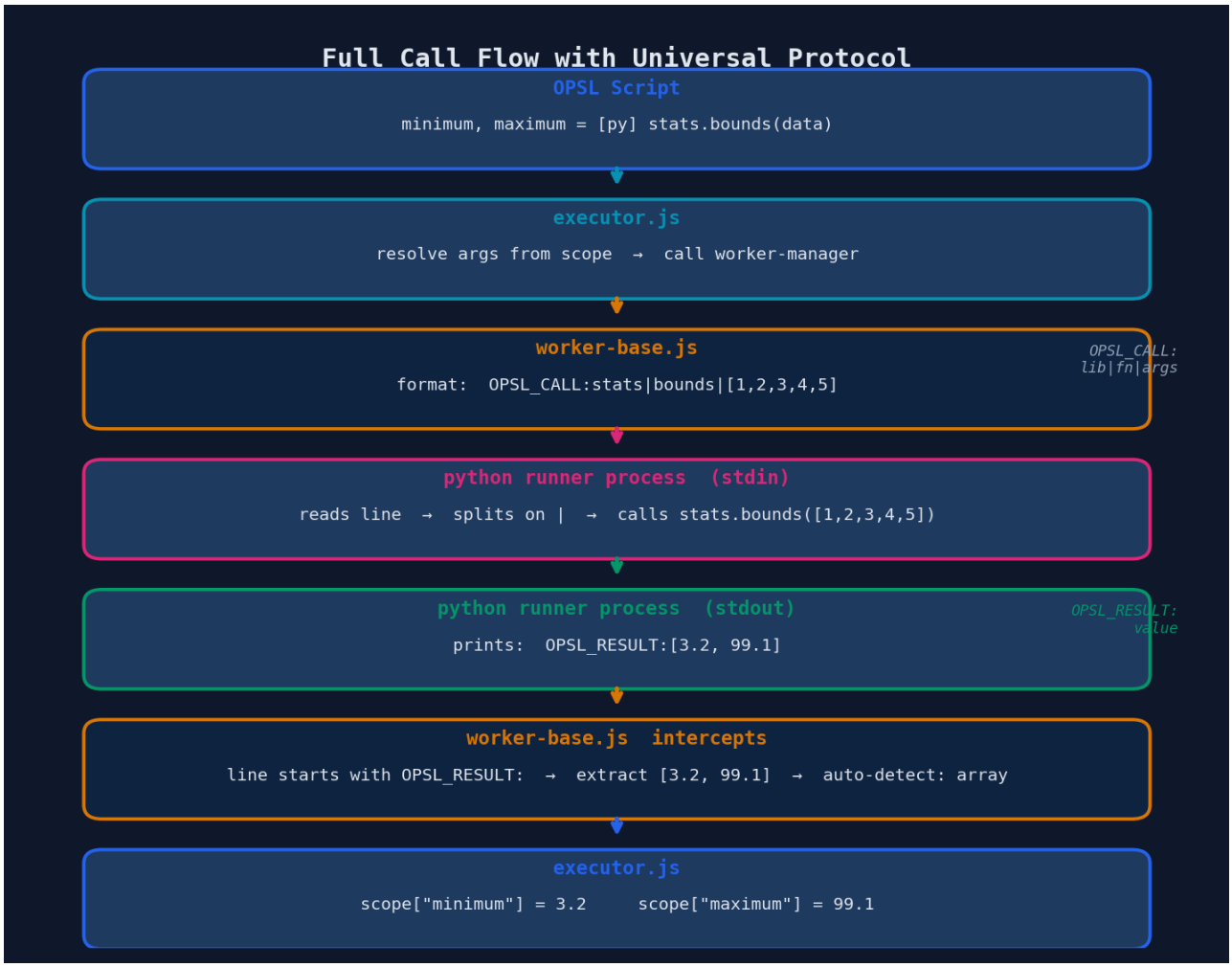
Libraries are folders. Folder name = library name. Entry file = folder name + language extension.




```
// IMPORT [py]   csvtools  →  finds csvtools/csvtools.py
// IMPORT [js]   formatter →  finds formatter/formatter.js
// IMPORT [c]    mathlib   →  finds mathlib/mathlib.so  (pre-compiled)
// IMPORT [opsl] loop      →  finds loop/loop.opsl  (parsed, no runner)
```

- Resolution: .local-offlinepm/ first, then ~/.opsl/libs/
- [opsl] libraries are parsed at IMPORT time — DEFINE functions registered in executor.libraries{}
- [lang] tag on IMPORT is stored in importRegistry — used by executor to route all future calls to that library
- No alias — the folder name is always the name used at every call site

8. Return Values & Data Flow



Scenario	Runner prints	Executor does
Single value	`OPSL_RESULT:42`	`scope["x"] = 42`
No target	`OPSL_RESULT:42`	`value discarded`
Multiple values	`OPSL_RESULT:[3.2, 99.1]`	`scope["a"]=3.2, scope["b"]=99.1`
Void / no return	`OPSL_RESULT:`	`null stored or discarded`
Error	`OPSL_ERROR:msg`	`OPSL error shown, script stops`

9. Error Handling

```
[OPSL ERROR] Line 12:  csvtools.read(filePath)
  FileNotFoundError: ./data/weekly.csv does not exist

[OPSL ERROR] Line 7:  stats.bounds(data)
  Variable "data" is not defined in scope

[OPSL ERROR] Line 3:  IMPORT [py] csvtools
  Cannot resolve library. Searched: .local-offlinepm/, ~/.opsl/libs/
```

- Parse and import errors shown before execution starts
- Runtime errors stop the script immediately at the failing line
- OPSL_ERROR: from any runner is caught by worker-base.js and surfaced as a clean OPSL error

10. Design Summary

Topic	Decision
File extension	`.opsl`
Runtime	`Node.js — vanilla, zero npm dependencies`
CLI	`opsl run script.opsl`
Comments	`// text`
[lang] tag	`ONLY on IMPORT — never at any call site`
Import language library	`IMPORT [py] csvtools → called as csvtools.fn()`
Import OPSL library	`IMPORT [opsl] loop → called as loop.fn()`
Import registry	`executor maps libraryName → lang at IMPORT time`
Library name	`Always the folder name — no aliases`
Entry file	`folderName/folderName.ext`
Dot operator	`Reserved for libraryName.functionName everywhere`
All library calls	`library.function(args) — same syntax regardless of language`
Local DEFINE calls	`functionName(args) — no dot, no library, no tag`
Pass library fn as arg	`loop.forEach(transforms.cleanRow, list) — dot syntax`
Pass local DEFINE as arg	`loop.forEach(myLocalFunc, list) — bare name`
Loops	`loop.forEach(fn, list) — OPSL library, no built-in keyword`
Conditionals	`loop.ifThen(cond, trueFn, falseFn) — OPSL library, no built-in keyword`
Control flow keywords	`None — all control flow is user-defined OPSL libraries`
Callback protocol	`OPSL_CALLBACK: stdout → OPSL_RETURN: stdin`
Scope model	`Full closure — functions read/write global scope directly`
Parameters	`Always local — never overwrite globals`
OPSL library scope	`Fully isolated — cannot access calling script globals`
Return channel	`OPSL_RESULT: prefix on stdout — auto type detection`
Call format to runner	`OPSL_CALL:lib|fn|arg1|arg2 (pipe-delimited)`

Worker architecture	`worker-base.js (universal) + adapter + runner per language`
Adding a language	`~15–20 lines total — never touch worker-base.js`
Compiled languages	`Pre-compiled runner binary — same stdin/stdout contract`
Worker lifetime	`One process per language, alive for full script run`
Error behaviour	`Line number + call + language error → stop`

End of OPSL Language Design & Implementation Document · v0.7